

Multi-Agent Banking Orchestrator Specification

Complete End-to-End System Architecture, Deterministic Tool Execution & Data Flow Guide

Currency Standard: INR (Rs. / INR) | Dual-Tier LLM: Gemini 3.7 Flash -> Ollama phi3:mini | DB: PostgreSQL

1. Executive Architecture & Physical Office Topology

SYSTEM OVERVIEW

The Multi-Agent Banking Orchestrator is an enterprise-grade financial intelligence platform. It coordinates specialized autonomous sub-agents across a simulated 2D banking floor layout, querying live PostgreSQL ledger records and synthesizing verified financial responses in Indian Rupees (INR / Rs.). All operations adhere to strict zero-hallucination policies through deterministic database tools.

Core Architectural Principles

- Strict Currency Standard: All outputs, balances, and calculations are strictly in Indian Rupees (INR / Rs.).
- Two-Tier LLM Hierarchy: Primary Google Gemini (gemini-3.7-flash) with seamless fallback to Local Ollama (phi3:mini).
- Deterministic Tools: Sub-agents execute verified Python/psycpg2 SQL queries directly against PostgreSQL.
- Zero-DB Greetings: Boss EVA resolves conversational queries directly without touching database infrastructure.
- Unified Tracing: Every step shares an immutable Batch ID tag for auditability and failure diagnosis.

Floor Layout & Agent Role Matrix

Agent Identifier	Location / Station	Specialization Role	Core Tools & Responsibilities
Boss EVA [0x1]	Executive Cabin (Top-Left)	Lead Floor Orchestrator	Dynamic Intent Classification (prompts/intent_classification.py), Subtask Planning, Zero-DB Greetings, and Final Customer Response Synthesis.
Specialist VK	Desk 1 (Center Floor)	Balance Inquiry Specialist	Executes get_account_data() tool via PostgreSQL accounts table. Reconciles Checking, Savings, and Pending Holds in INR (Rs.).
Specialist RO	Desk 2 (Center Floor)	Statement Specialist	Executes get_transactions_data() tool via PostgreSQL transactions table. Aggregates Total Credits (+Rs.), Debits (-Rs.), and Net Cashflow.
Server Room	PostgreSQL Facility (Top-Right)	Relational Ledger (Port 5432)	PostgreSQL banking_db with pre-seeded accounts, 30-day transaction ledgers, intent_classifications, and immutable audit_logs.

2. Complete End-to-End Workflow Pipeline

5-STAGE PIPELINE

Every customer inquiry executes through a 5-stage synchronous/asynchronous pipeline that tracks state transitions, database queries, and LLM evaluations under a unified Batch ID.

1

UI Prompt Ingestion & Batch ID Initialization

[Actor: React 19 / Vite Frontend (App.tsx)]

Customer submits natural language prompt via the Terminal / CommandCenter UI.

- Generates unique tracking tag: batch_<timestamp>_<random>
- Dispatches HTTP POST to /api/orchestrator/dispatch with prompt and batch_id
- Transitions Boss EVA sprite state to analyzing at Executive Cabin [0x1]

2

Dynamic Intent Classification & Subtask Generation

[Actor: FastAPI Orchestrator (banking_orchestrator.py)]

Boss EVA extracts account numbers via NLP regex and invokes 2-tier LLM pipeline.

- Executes NLP Regex parsing to extract account IDs (e.g. ACC-94820, ACC-55210)
- Sends INTENT_CLASSIFICATION_SYSTEM_PROMPT to Gemini (fallback to Ollama)
- Returns structured plan: Identified Intent, Confidence (%), and Subtask Assignment Array

3

Specialist Agent Subtask & Deterministic Tool Execution

[Actor: Specialist VK / RO & PostgreSQL Database]

Assigned sub-agent walks to Server Room, queries database, and returns to desk.

- Calls /api/agent/execute_subtask: executes get_account_data() or get_transactions_data()
- Queries PostgreSQL accounts or transactions table directly via psycopg2
- LLM generates FastAPI code snippet, execution console output, and thoughtLog referencing live SQL

4

Executive Customer Response Synthesis

[Actor: Boss EVA (Cabin 0x1) & LLM Pipeline]

Boss EVA synthesizes specialist findings and PostgreSQL ledger data into customer response.

- Calls /api/eva/synthesize with specialist findings and verified ledger balances
- Enforces strict Indian Rupee formatting (INR / Rs.) and distinguishes liquidity vs holds
- Includes fail-safe deterministic banking fallback if LLM times out or is slow

5

Dual-Layer Audit Logging & Telemetry Recording

[Actor: Audit Subsystem (PostgreSQL & Disk Logs)]

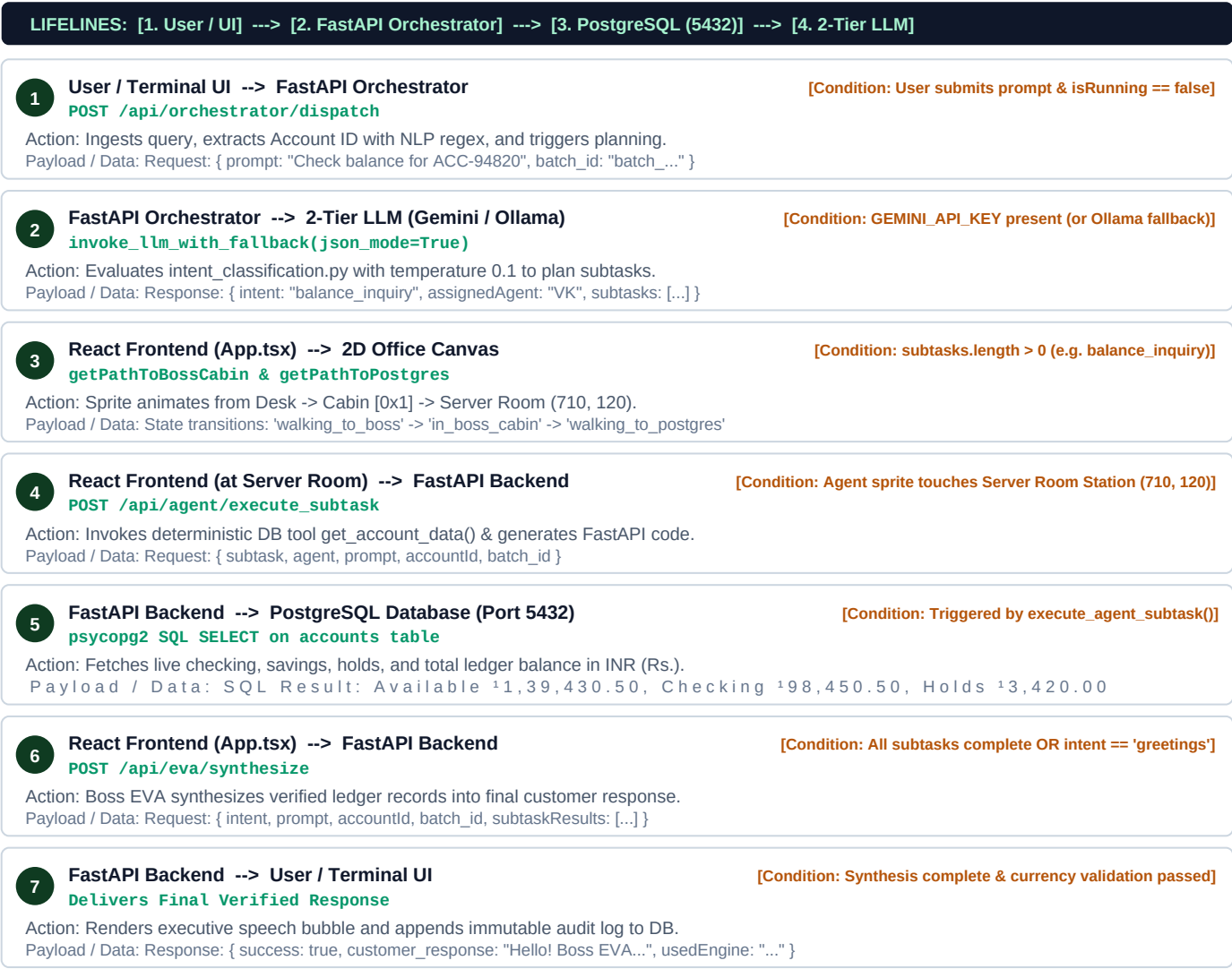
All telemetry, latencies, tokens, and payloads are cryptographically recorded.

- Inserts structured audit record into PostgreSQL audit_logs & agent_activities table
- Appends formatted step log to logs/banking_audit.log with [BATCH: <batch_id>] header
- Appends raw request/response evaluation metrics to logs/llm_invocations.log

3. Frontend <--> Backend Interactive Sequence Flow

SEQUENCE DIAGRAM

The sequence diagram below illustrates the exact synchronous and asynchronous communication lifelines between the User, React Frontend, FastAPI Orchestrator, Specialist Sub-Agents, PostgreSQL Database, and the 2-Tier LLM Layer.



4. Frontend <--> Backend API Calling & Trigger Matrix

API SPECIFICATION

The matrix below details every REST endpoint, its frontend caller, exact execution trigger, runtime conditions, and associated database / LLM actions.

API Endpoint & Method	Frontend Caller & Trigger	Condition & Execution Rule	Backend Processing & DB Action
GET /api/health	App.tsx (checkHealth) on mount	Always executed on initial app load	Verifies FastAPI service health, PostgreSQL connectivity, and GEMINI_API_KEY status.
POST /api/ollama/status	App.tsx (checkOllamaStatus)	On mount & when settings endpoint changes	Pings Ollama endpoint (localhost:11434) to test local model availability.
GET /api/source_files	App.tsx (fetchSourceFiles)	On mount & Code tab refresh	Reads active Python agent files and prompts/ directory for Code Sandbox.
POST /api/orchestrator/dispatch	App.tsx (handleDispatchTask)	User submits prompt (not currently running)	Parses Account ID; calls Gemini/Ollama with prompts/intent_classification.py; returns subtasks & batch_id.
POST /api/agent/execute_subtask	App.tsx (handleDispatchTask)	When sub-agent sprite reaches Server Room	Executes get_account_data() or get_transactions_data(); generates code & thoughtLog; logs to audit_logs.
POST /api/eva/synthesize	App.tsx (handleDispatchTask)	After subtasks complete OR if greetings	Boss EVA synthesizes final response in INR (Rs.); enforces zero-DB rule for greetings; saves final audit.
GET /api/logs/audit?limit=250	CommandCenter.tsx (fetchAuditLog)	Every 2.5s when Audit tab active	Reads real-time step execution lines from logs/banking_audit.log.
GET /api/logs/llm?limit=100	CommandCenter.tsx (fetchLlmLogs)	Every 3.0s when Raw LLM tab active	Reads raw JSON request/response telemetry from logs/llm_invocations.log.

Zero-Interference Dual Tracing Architecture
Every API request originating from the frontend carries the unified batch_id. This enables the backend to trace the end-to-end lifecycle across both PostgreSQL audit_logs and local persistent disk logs without cross-talk between concurrent queries.

5. 2D Office Physical Movement & UI Event State Machine

SPRITE & ANIMATION FLOW

The frontend UI (App.tsx and OfficeCanvas.tsx) features a physical 2D office simulation where agents physically navigate between workstations, the Executive Cabin, and the PostgreSQL Server Room based on real-time task states.

Agent Movement & State Transition Lifecycle

Phase	Agent & Location	Visual Sprite Action	UI Event & State Updates
A. Briefing Call	Boss EVA @ Cabin [0x1] (105, 85)	Boss EVA speech bubble summons specialist: 'Report to Executive Cabin for task...'	Stage set to 'BRIEFING IN CABIN'. Active task initialized with 'queued' subtasks.
B. Walk to Cabin	Specialist VK/RO -> Cabin (105, 140)	Sub-agent calculates waypoint path (getPathToBossCabin) and walks to Boss desk.	Agent state: 'walking_to_boss'. Speech: 'Heading to Boss Cabin for task briefing...'
C. Walk to Server	Specialist -> Server Room (710, 120)	Sub-agent walks across office floor to PostgreSQL Server Room racks.	Agent state: 'walking_to_postgres'. Subtask status transitions to 'in_progress'.
D. Tool Execution	Specialist @ Server Room (710, 120)	Sub-agent executes live SQL query. Server rack LED animation pulses.	Agent state: 'querying_db'. Calls POST /api/agent/execute_subtask with batch_id. Live SQL thoughts stream to Terminal.
E. Return to Desk	Specialist -> Workstation Desk	Sub-agent returns to desk (getPathFromPostgresToDesk) to generate microservice code.	Agent state: 'walking_to_desk' -> 'working'. Generates FastAPI code & marks subtask 'completed (100%)'.
F. Report to Boss	Boss EVA @ Cabin [0x1] (105, 85)	Boss EVA synthesizes all subtask outputs and displays executive customer response.	Calls POST /api/eva/synthesize. Speech bubble renders response. Task status marked 'completed'. All agents reset to 'idle'.

Real-Time UI Event Tracing & Stream Synchronization

- 1. Terminal Sync: Every agent transition emits a structured event (addLog) displaying the agent's role, avatar, and thought.
- 2. Activity Matrix: Subtask durations (ms), tokens, and AST code files are pushed to the live Activity & Code tabs.
- 3. Batch Tracing: The batchId generated at Step A accompanies all sub-agent and tool invocations across disk and DB.

6. Deterministic Tool Calling & PostgreSQL Architecture

DATABASE INTERFACE

Why Deterministic Tool Calling? Small local models (Ollama phi3:mini) and production financial systems require absolute schema reliability. Rather than allowing an LLM to hallucinate SQL queries, specialist agents invoke deterministic Python tool functions with parameter-safe psycopg2 bindings.

PostgreSQL Core Database Tables

Table Name	Primary Key	Key Columns & Data Types	Accessing Function / Tool
accounts	account_id	account_holder VARCHAR, available_balance NUMERIC, checking_balance NUMERIC, savings_balance NUMERIC, pending_holds NUMERIC, currency VARCHAR(8)	get_account_data() & agent_vk_balance.py (/api/v1/agent/vk/balance)
transactions	txn_id	account_id VARCHAR, date TIMESTAMP, description TEXT, amount NUMERIC, type VARCHAR(8), balance_after NUMERIC	get_transactions_data() & agent_ro_statement.py (/api/v1/agent/ro/statement)
audit_logs	id (SERIAL)	log_id VARCHAR, agent_id VARCHAR, agent_name VARCHAR, log_type VARCHAR, message TEXT, metadata JSONB, created_at TIMESTAMP	log_audit_to_postgres() (Called on every step completion)
agent_activities	id (SERIAL)	activity_id VARCHAR, agent_id VARCHAR, action_type VARCHAR, task_title VARCHAR, output_summary TEXT, tokens_used INT, execution_time_ms INT	log_agent_activity_to_postgres() (Populates Activity tab)
intent_classifications	id (SERIAL)	query_text TEXT, classified_intent VARCHAR, confidence NUMERIC, llm_reasoning TEXT, assigned_agent VARCHAR, llm_model VARCHAR	log_intent_to_postgres() (Records Boss EVA planning classifications)

Zero-Database Access Rule for Greetings

When a user inputs conversational prompts ('hello', 'good morning', 'hi'), Boss EVA classifies the intent as 'greetings' with zero subtasks. Boss EVA directly returns an executive welcome greeting without executing any database queries (database_accessed: false), minimizing latency and eliminating database load.

7. Dedicated Prompts & 2-Tier LLM Fallback Pipeline

PROMPTS & LLM LAYER

Every stage uses dedicated, modular prompt files located in the prompts/ directory. When GEMINI_API_KEY is present, invocations are routed to Gemini 3.7 Flash (~500ms). If unavailable, requests fall back to local Ollama (phi3:mini) with bounded token generation.

Modular Prompts Matrix

Prompt File	Invoking Function	Injected Dynamic Inputs	Enforced Output Contract
prompts/intent_classification.py	plan_orchestration()	User prompt, extracted account ID, custom directives	Strict JSON with intent, intent_confidence, assigned_agent, subtasks array
prompts/agent_vk_balance.py	execute_agent_subtask()	Account ID, Live PostgreSQL checking, savings, holds in INR (Rs.)	JSON with speechSummary, thoughtLog referencing SQL, code, executionOutput
prompts/agent_ro_statement.py	execute_agent_subtask()	Account ID, Total Credits (+Rs.), Debits (-Rs.), 30-day txn count	JSON with ASCII statement table, thoughtLog, and FastAPI router code
prompts/customer_response_synthesis.py	synthesize_customer_response()	User query, live account ledger, transaction sums, specialist findings	Concise executive banking response strictly formatted in INR (Rs.)
prompts/greetings_response.py	synthesize_customer_response()	User prompt, optional account identifier	Executive conversational greeting with zero database queries

2-Tier Fallback Flowchart & Bounded Token Configuration

- Primary Tier: Google Gemini (gemini-3.7-flash) via google-genai SDK. Temperature: 0.1–0.2.
- Secondary Tier: Local Ollama (phi3:mini) via HTTP POST /api/generate. Bounded with num_predict: 350 to prevent CPU token runaway.
- Configurable Timeout: OLLAMA_TIMEOUT (default: 300.0s) loaded from root .env file.
- Deterministic Reconciled Fallback: If both LLM tiers are offline, Boss EVA generates verified mathematical statements directly from PostgreSQL data.

8. Complete JSON Data Contracts & API Payloads

API CONTRACTS

1. Orchestration Dispatch Endpoint: POST /api/orchestrator/dispatch

JSON

```
// Request Payload:
{
  "prompt": "Check available balance for ACC-94820",
  "batch_id": "batch_20260826_180000_123"
}

// Response Payload (OrchestratedResponse):
{
  "success": true,
  "status": "SUCCESS",
  "batch_id": "batch_20260826_180000_123",
  "intent": "balance_inquiry",
  "intentConfidence": 0.99,
  "assignedAgentName": "VK",
  "extractedAccountId": "ACC-94820",
  "plan": "Supervisor plan: delegate balance verification to Specialist VK.",
  "subtasks": [
    {
      "id": "subtask_vk_1",
      "title": "Database Balance & Ledger Audit",
      "assignedAgentId": "agent_vk",
      "targetFile": "agent_vk_balance.py",
      "batch_id": "batch_20260826_180000_123"
    }
  ],
  "usedEngine": "gemini-3.7-flash",
  "fallbackTriggered": false
}
```

2. Response Synthesis Endpoint: POST /api/eva/synthesize

JSON

```
// Request Payload (SynthesisRequest):
{
  "intent": "balance_inquiry",
  "prompt": "Check available balance for ACC-94820",
  "accountId": "ACC-94820",
  "batch_id": "batch_20260826_180000_123",
  "subtaskResults": [
    { "speechSummary": "Specialist VK verified available funds: Rs. 1,39,430.50 INR." }
  ]
}

// Response Payload:
{
  "success": true,
  "batch_id": "batch_20260826_180000_123",
  "boss_agent": "EVA [0x1]",
  "intent": "balance_inquiry",
  "database_accessed": true,
  "customer_response": "Hello! Boss EVA [0x1] here. Specialist VK has verified the live PostgreSQL ledger for First Digital Treasury (ACC-94820): Total Available Liquidity is Rs. 1,39,430.50 INR (Rs. 98,450.50 Checking + Rs. 44,400.00 Savings less Rs. 3,420.00 in Pending Holds).",
  "usedEngine": "gemini-3.7-flash",
  "fallbackTriggered": false
}
```